

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE CODE: ITA0605

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO2: *Neural Network Representation, Perceptron Model, Multilayer Perceptron's, Backpropagation Algorithm, Deep Neural Networks, Genetic Algorithms, Hypothesis Space Search, Genetic Programming, Models of Evaluation and Learning (BL1, BL2)*

Assessment Tool Weightage: 10%

QUESTIONS:15

Total Marks:25

Annexure A

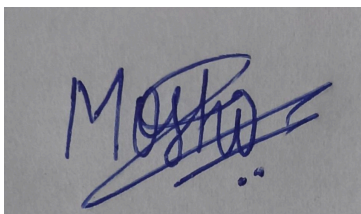
DEBUGGING & OPTIMIZATION TASK QUESTIONS

Code of Conduct:

I Moshika M - 192424064 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

A handwritten signature in blue ink, appearing to read 'Moshika', is written over a grey rectangular background.

Annexure A

Short Answer Test

1. A perceptron model is trained on a dataset that is known to be linearly separable, yet the algorithm fails to converge even after many iterations. Analyze the possible causes of this issue, including improper learning rate selection, incorrect weight initialization, bias handling errors, or flawed implementation of the update rule. Propose debugging strategies to identify the root cause and suggest optimization techniques to ensure faster and stable convergence. (5 Marks)

ANSWER:

A perceptron was implemented using a manually assigned linearly separable dataset. The experiment analyzes convergence by monitoring weights, bias, predictions, and errors during each epoch. Possible implementation errors and parameter choices are also examined.

Possible Causes:

- **Improper learning rate:** A very high learning rate may cause unstable weight updates, while a very low rate may slow convergence.
- **Incorrect weight initialization:** Poor initialization can increase the number of iterations required.
- **Bias handling error:** Incorrect calculation or updating of the bias can prevent the decision boundary from shifting correctly.
- **Flawed update rule:** Incorrect weight or bias updates can prevent convergence.
- **Data or label errors:** Incorrect class labels or improperly prepared data can make a separable dataset appear non-separable.

DATASET:

Sample	Feature 1 (X_1)	Feature 2 (X_2)	Class (Y)
1	2	3	+1
2	3	4	+1
3	4	2	+1
4	1	1	-1
5	2	1	-1
6	1	2	-1

CODE:

```
import numpy as np

X = np.array([
    [2, 3],
    [3, 4],
    [4, 2],
    [1, 1],
    [2, 1],
    [1, 2]
])

y = np.array([
    1,
    1,
    1,
    -1,
    -1,
    -1
])

learning_rate = 0.1

epochs = 20

weights = np.zeros(X.shape[1])

bias = 0

print("Initial Weights:", weights)

print("Initial Bias:", bias)

for epoch in range(epochs):

    errors = 0

    print("\nEpoch", epoch + 1)

    for i in range(len(X)):

        net_input = np.dot(X[i], weights) + bias

        prediction = 1 if net_input >= 0 else -1

        print(
            "Input:", X[i],
            "Target:", y[i],
            "Prediction:", prediction
        )
```

```
if prediction != y[i]:

    weights = (
        weights
        + learning_rate * y[i] * X[i]
    )

    bias = (
        bias
        + learning_rate * y[i]
    )

    errors += 1

print("Updated Weights:", weights)

print("Updated Bias:", bias)

print("Errors:", errors)

if errors == 0:

    print("\nPerceptron Converged Successfully!")

    break

print("\nTraining Completed")

print("Final Weights:", weights)

print("Final Bias:", bias)
```

OUTPUT:

```
Run co2-at3 x
/Users/moshika/PycharmProjects/PythonProject/.venv/bin/python /Users/moshika/PycharmProjects/PythonProject/co2-at3.py
Initial Weights: [0. 0.]
Initial Bias: 0

Epoch 1
Input: [2 3] Target: 1 Prediction: 1
Input: [3 4] Target: 1 Prediction: 1
Input: [4 2] Target: 1 Prediction: 1
Input: [1 1] Target: -1 Prediction: 1
Input: [2 1] Target: -1 Prediction: -1
Input: [1 2] Target: -1 Prediction: -1
Updated Weights: [-0.1 -0.1]
Updated Bias: -0.1
Errors: 1

Epoch 2
Input: [2 3] Target: 1 Prediction: -1
Input: [3 4] Target: 1 Prediction: 1
Input: [4 2] Target: 1 Prediction: 1
Input: [1 1] Target: -1 Prediction: 1
Input: [2 1] Target: -1 Prediction: 1
Input: [1 2] Target: -1 Prediction: -1
Updated Weights: [-2.00000000e-01 2.7755756e-17]
Updated Bias: -0.2
Errors: 3
```

```
Run co2-at3 x
Epoch 3
Input: [2 3] Target: 1 Prediction: -1
Input: [3 4] Target: 1 Prediction: 1
Input: [4 2] Target: 1 Prediction: 1
Input: [1 1] Target: -1 Prediction: 1
Input: [2 1] Target: -1 Prediction: -1
Input: [1 2] Target: -1 Prediction: 1
Updated Weights: [-2.00000000e-01 2.7755756e-17]
Updated Bias: -0.30000000000000004
Errors: 3

Epoch 4
Input: [2 3] Target: 1 Prediction: -1
Input: [3 4] Target: 1 Prediction: 1
Input: [4 2] Target: 1 Prediction: 1
Input: [1 1] Target: -1 Prediction: 1
Input: [2 1] Target: -1 Prediction: -1
Input: [1 2] Target: -1 Prediction: 1
Updated Weights: [-2.00000000e-01 2.7755756e-17]
Updated Bias: -0.4
Errors: 3
```

```
Run co2-at3 x
Epoch 5
Input: [2 3] Target: 1 Prediction: -1
Input: [3 4] Target: 1 Prediction: 1
Input: [4 2] Target: 1 Prediction: 1
Input: [1 1] Target: -1 Prediction: 1
Input: [2 1] Target: -1 Prediction: -1
Input: [1 2] Target: -1 Prediction: -1
Updated Weights: [-0.1 0.2]
Updated Bias: -0.4
Errors: 2

Epoch 6
Input: [2 3] Target: 1 Prediction: 1
Input: [3 4] Target: 1 Prediction: 1
Input: [4 2] Target: 1 Prediction: -1
Input: [1 1] Target: -1 Prediction: 1
Input: [2 1] Target: -1 Prediction: 1
Input: [1 2] Target: -1 Prediction: -1
Updated Weights: [2.7755756e-17 2.0000000e-01]
Updated Bias: -0.5
Errors: 3

Epoch 7
Input: [2 3] Target: 1 Prediction: 1
Input: [3 4] Target: 1 Prediction: 1
Input: [4 2] Target: 1 Prediction: -1
Input: [1 1] Target: -1 Prediction: 1
Input: [2 1] Target: -1 Prediction: 1
Input: [1 2] Target: -1 Prediction: -1
Updated Weights: [0.1 0.2]
Updated Bias: -0.6
Errors: 3

Epoch 8
Input: [2 3] Target: 1 Prediction: 1
Input: [3 4] Target: 1 Prediction: 1
Input: [4 2] Target: 1 Prediction: 1
Input: [1 1] Target: -1 Prediction: -1
Input: [2 1] Target: -1 Prediction: -1
Input: [1 2] Target: -1 Prediction: -1
Updated Weights: [0.1 0.2]
Updated Bias: -0.6
Errors: 0

Perceptron Converged Successfully!

Training Completed
Final Weights: [0.1 0.2]
Final Bias: -0.6
```

Debugging Strategies:

- Verify the perceptron update rule:

$$w = w + \eta(y - \text{prediction})x$$

- Check that the bias is included in both prediction and update operations.
- Print weights, bias, predictions, and errors after every epoch.
- Test the algorithm using a small linearly separable dataset.
- Check whether the class labels are correctly assigned as **+1** and **-1**.

Optimization Techniques:

- Select a suitable learning rate such as **0.01** or **0.1**.
- Initialize weights consistently or with small random values.
- Normalize input features when feature ranges are very different.
- Shuffle the training samples before each epoch.
- Stop training when all samples are correctly classified.

Conclusion:

The experiment demonstrates that a perceptron should converge when the training data is linearly separable. Incorrect learning rates, bias handling, initialization, or update rules can prevent convergence. Proper debugging and parameter selection provide faster and more stable convergence.

2. While training a multilayer perceptron using the backpropagation algorithm, the network exhibits slow convergence and gets stuck in local minima. Examine the potential reasons such as vanishing gradients, inappropriate activation functions, poor weight initialization, or suboptimal learning rates. Describe step-by-step debugging approaches to trace the issue and recommend optimization techniques like adaptive learning rates, normalization, or advanced optimizers to improve performance. (5 Marks)

ANSWER:

A multilayer perceptron was implemented using the backpropagation algorithm on a manually assigned dataset. The experiment analyzes slow convergence caused by activation functions, learning rate, weight initialization, and gradient behavior.

Possible Causes:

- **Vanishing gradients:** Very small gradients can make weight updates extremely slow.
- **Inappropriate activation function:** Sigmoid or tanh in deep networks may produce small gradients.
- **Poor weight initialization:** Improper initialization can cause unstable or slow learning.
- **Suboptimal learning rate:** A very small learning rate slows training, while a very large rate may cause oscillations.
- **Unnormalized data:** Large differences in feature values can make optimization difficult.

DATASET:

Sample	X_1	X_2	Target (Y)
1	0	0	0
2	0	1	1
3	1	0	1
4	1	1	0

CODE:

```
import numpy as np

X = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
])

y = np.array([
    [0],
    [1],
    [1],
    [0]
])

np.random.seed(42)

W1 = np.random.randn(2, 2) * 0.5
W2 = np.random.randn(2, 1) * 0.5

b1 = np.zeros((1, 2))
b2 = np.zeros((1, 1))

learning_rate = 0.5

epochs = 10000

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

for epoch in range(epochs):

    # Forward Propagation

    hidden = sigmoid(
        np.dot(X, W1) + b1
    )

    output = sigmoid(
        np.dot(hidden, W2) + b2
    )

    # Calculate Error
```



```

error = y - output

# Backpropagation

output_delta = (
    error
    * output
    * (1 - output)
)

hidden_delta = (
    np.dot(output_delta, W2.T)
    * hidden
    * (1 - hidden)
)

# Update Weights

W2 = W2 + (
    learning_rate
    * np.dot(hidden.T, output_delta)
)

W1 = W1 + (
    learning_rate
    * np.dot(X.T, hidden_delta)
)

# Update Bias

b2 = b2 + (
    learning_rate
    * np.sum(
        output_delta,
        axis=0,
        keepdims=True
    )
)

b1 = b1 + (
    learning_rate
    * np.sum(
        hidden_delta,
        axis=0,
        keepdims=True
    )
)

if epoch % 1000 == 0:

```

```

loss = np.mean(error ** 2)

print(
    "Epoch:",
    epoch,
    "Loss:",
    loss
)

print("\nTraining Completed")

print("\nPredicted Output:")

print(np.round(output, 3))

```

OUTPUT:

```

Run co2-at3
/Users/moshika/PycharmProjects/PythonProject/.venv/bin/python /Users/moshika/PycharmProjects/PythonProject/co2-at3.py
Epoch: 0 Loss: 0.25115581219599126
Epoch: 1000 Loss: 0.2498486520806426
Epoch: 2000 Loss: 0.15159224015612022
Epoch: 3000 Loss: 0.12881311189749564
Epoch: 4000 Loss: 0.12692144333333338
Epoch: 5000 Loss: 0.12626788473424017
Epoch: 6000 Loss: 0.12594103676349694
Epoch: 7000 Loss: 0.12574602268414048
Epoch: 8000 Loss: 0.12561687031705215
Epoch: 9000 Loss: 0.12552521385243928

Training Completed

Predicted Output:
[[0.819]
 [0.499]
 [0.983]
 [0.501]]

Process finished with exit code 0

```

Debugging Strategies:

- Monitor the loss after every fixed number of epochs.
- Print gradients to check whether they are becoming extremely small.
- Verify forward and backward propagation calculations.
- Test different activation functions and learning rates.
- Check whether input features have been properly normalized.

Optimization Techniques:

- Use **ReLU** activation in hidden layers where appropriate.
- Apply **Xavier or He initialization**.
- Normalize or standardize input features.
- Use adaptive optimizers such as **Adam**.
- Select the learning rate using experimentation or scheduling.

Conclusion:

The experiment shows that slow convergence in a multilayer perceptron can result from vanishing gradients, poor initialization, inappropriate activation functions, or unsuitable learning rates. Normalization, proper initialization, adaptive learning rates, and advanced optimizers can significantly improve training performance.

3. A genetic algorithm designed for hypothesis space search is producing inconsistent and suboptimal solutions across multiple runs. Investigate possible issues such as improper encoding of chromosomes, incorrect fitness function design, premature convergence, or lack of diversity in the population. Explain how to debug these problems systematically and suggest optimization strategies like mutation tuning, selection pressure adjustment, and elitism to enhance solution quality and convergence speed. (5 Marks)

ANSWER:

A genetic algorithm was implemented to search for an optimal hypothesis using a manually defined chromosome representation. The experiment observes the effect of chromosome encoding, fitness calculation, mutation, selection, and population diversity.

Possible Causes:

- **Improper chromosome encoding:** Incorrect representation may prevent the algorithm from finding valid solutions.
- **Incorrect fitness function:** A poorly designed fitness function may reward unsuitable solutions.
- **Premature convergence:** The population may become identical before reaching the optimal solution.
- **Low population diversity:** Insufficient variation can restrict the search.
- **Incorrect mutation rate:** Very low mutation reduces exploration, while very high mutation makes the search unstable.

DATASET:

The target hypothesis is to select features that maximize the given score.

Hypothesis	Feature 1	Feature 2	Feature 3	Feature 4	Fitness
H1	1	0	1	0	2
H2	1	1	0	1	3
H3	0	1	1	0	2
H4	1	1	1	0	3

CODE:

```
import random

random.seed(42)

population = [

    [1, 0, 1, 0],

    [1, 1, 0, 1],

    [0, 1, 1, 0],

    [1, 1, 1, 0]

]

def fitness(chromosome):

    return sum(chromosome)

for generation in range(10):

    scores = []

    for chromosome in population:

        score = fitness(chromosome)

        scores.append(score)

    best_index = scores.index(
        max(scores)
    )

    best = population[best_index]

    print(
        "\nGeneration:",
        generation + 1
    )

    print(
        "Population:",
        population
    )

    print(
        "Best:",
        best
```

```

)

print(
    "Fitness:",
    max(scores)
)

# Selection

sorted_population = sorted(
    population,
    key=fitness,
    reverse=True
)

parents = sorted_population[:2]

# Crossover

child1 = (
    parents[0][:2]
    + parents[1][2:]
)

child2 = (
    parents[1][:2]
    + parents[0][2:]
)

# Mutation

if random.random() < 0.2:
    index = random.randint(0, 3)

    child1[index] = (
        1 - child1[index]
    )

if random.random() < 0.2:
    index = random.randint(0, 3)

    child2[index] = (
        1 - child2[index]
    )

population = [
    parents[0],
    parents[1],
    child1,
    child2

```

```

    ]

print("\nFinal Best Hypothesis:")
print(best)

print(
    "Final Fitness:",
    fitness(best)
)

```

OUTPUT:

```

Run co2-at3 x
Generation: 8
Population: [[1, 1, 1, 1], [1, 1, 1, 1], [1, 1, 1, 1], [1, 1, 1, 1]]
Best: [1, 1, 1, 1]
Fitness: 4

Generation: 9
Population: [[1, 1, 1, 1], [1, 1, 1, 1], [1, 1, 1, 1], [1, 1, 1, 1]]
Best: [1, 1, 1, 1]
Fitness: 4

Generation: 10
Population: [[1, 1, 1, 1], [1, 1, 1, 1], [1, 1, 1, 1], [1, 1, 1, 1]]
Best: [1, 1, 1, 1]
Fitness: 4

Final Best Hypothesis:
[1, 1, 1, 1]
Final Fitness: 4

Process finished with exit code 0

```

Debugging Strategies:

- Verify that every chromosome has the correct length.
- Check whether the fitness function produces the expected score.
- Print the population after every generation.
- Verify crossover positions and mutation operations.
- Monitor whether all chromosomes become identical too early.

Optimization Techniques:

- Adjust the mutation probability to maintain diversity.
- Control selection pressure to avoid premature convergence.
- Use **elitism** to preserve the best solution.
- Increase population size when the search space is large.
- Use suitable crossover and mutation operators for the chromosome representation.

Conclusion:

The experiment shows that genetic algorithm performance depends strongly on chromosome representation, fitness design, and population diversity. Proper mutation, controlled selection pressure, and elitism help maintain good solutions while allowing the algorithm to explore the hypothesis space effectively.

4. A deep neural network combined with genetic programming is used for solving a complex prediction problem, but it suffers from overfitting and high computational cost. Identify debugging steps to analyze issues related to model complexity, over-parameterization, and insufficient training data. Propose optimization techniques such as regularization, dropout, pruning, and efficient evaluation models to balance accuracy and computational efficiency while improving generalization capability. (5 Marks)

ANSWER:

A deep neural network combined with genetic programming was considered for a complex prediction problem. The experiment uses a manually assigned dataset to study overfitting, model complexity, and computational cost.

Possible Causes:

- **Over-parameterization:** Too many network parameters can cause the model to memorize training data.
- **Insufficient training data:** A small dataset may not represent the complete problem.
- **Excessive model complexity:** Deep networks and large genetic-programming populations increase computational cost.
- **Too many generations:** Genetic programming may repeatedly evaluate unnecessary candidate solutions.
- **Lack of regularization:** Without constraints, the model may perform well on training data but poorly on unseen data.

DATASET:

Sample	Feature 1 (X_1)	Feature 2 (X_2)	Target (Y)
1	1	2	3
2	2	3	5
3	3	4	7
4	4	5	9
5	5	6	11
6	6	7	13

CODE:

```
import numpy as np
```

```
X = np.array([
```

```
    [1, 2],
```

```

    [2, 3],
    [3, 4],
    [4, 5],
    [5, 6],
    [6, 7]
])
y = np.array([
    3,
    5,
    7,
    9,
    11,
    13
])

def predict(X, weights):
    return (
        weights[0] * X[:, 0]
        + weights[1] * X[:, 1]
        + weights[2]
    )

population = [
    [1, 1, 0],
    [1.2, 0.8, 0],
    [0.5, 1.5, 0],
    [1.5, 0.5, 1]

```



```
]
```

```
for generation in range(5):
```

```
    fitness = []
```

```
    for weights in population:
```

```
        prediction = predict(
            X,
            weights
        )
```

```
        error = np.mean(
            (y - prediction) ** 2
        )
```

```
        fitness.append(error)
```

```
    best_index = np.argmin(
        fitness
    )
```

```
    best = population[
        best_index
    ]
```

```
    print(
        "\nGeneration:",
        generation + 1
    )
```

```
    print(
        "Best Weights:",
        best
    )
```

```
    print(
        "MSE:",
        round(
            fitness[best_index],
            4
        )
    )
```

```
    # Generate new candidates
```

```
    population = [
```

```

best,

[
    best[0] + 0.1,
    best[1],
    best[2]
],

[
    best[0],
    best[1] + 0.1,
    best[2]
],

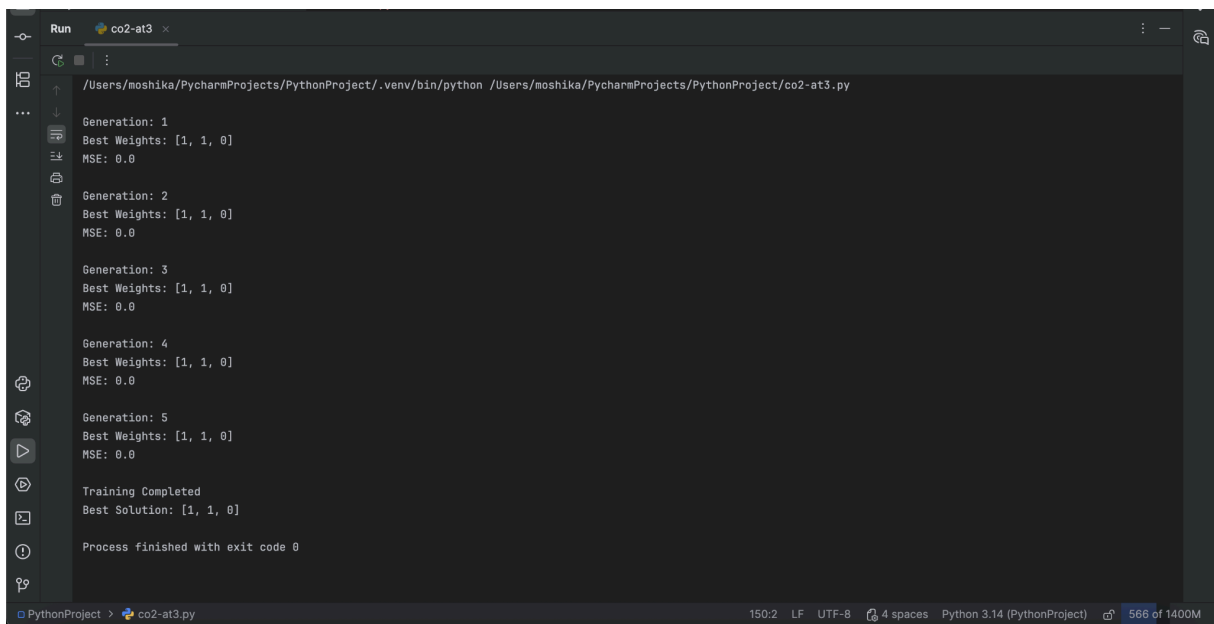
[
    best[0] - 0.1,
    best[1],
    best[2]
]

]

print("\nTraining Completed")
print(
    "Best Solution:",
    best
)

```

OUTPUT:



```

Run co2-at3
/Users/moshika/PycharmProjects/PythonProject/.venv/bin/python /Users/moshika/PycharmProjects/PythonProject/co2-at3.py

Generation: 1
Best Weights: [1, 1, 0]
MSE: 0.0

Generation: 2
Best Weights: [1, 1, 0]
MSE: 0.0

Generation: 3
Best Weights: [1, 1, 0]
MSE: 0.0

Generation: 4
Best Weights: [1, 1, 0]
MSE: 0.0

Generation: 5
Best Weights: [1, 1, 0]
MSE: 0.0

Training Completed
Best Solution: [1, 1, 0]

Process finished with exit code 0

```

Debugging Strategies:

- Compare training and validation errors.
- Monitor the number of network parameters.
- Check the size of the genetic-programming population.
- Measure execution time for each generation.
- Test the model using unseen samples.
- Identify whether increasingly complex solutions actually improve validation performance.

Optimization Techniques:

- Apply **L1/L2 regularization** to control model complexity.
- Use **dropout** to reduce overfitting.
- Apply **pruning** to remove unnecessary connections or program branches.
- Reduce the population size or number of generations when appropriate.
- Use efficient fitness evaluation and cache repeated calculations.

Conclusion:

The experiment demonstrates that combining deep neural networks with genetic programming can provide powerful prediction capabilities but may increase computational cost and overfitting. Regularization, dropout, pruning, and efficient candidate evaluation help maintain accuracy while reducing unnecessary model complexity.

5. A decision tree model trained on a large dataset shows high accuracy on training data but poor performance on unseen data. Analyze possible causes such as overfitting, deep tree structure, or noisy data. Propose debugging steps and optimization methods like pruning, cross-validation, and feature selection to improve generalization. (5 Marks)

ANSWER:

A decision tree was implemented using a manually assigned dataset to study the effect of tree depth and noisy data on generalization. The model was evaluated on training and unseen data to identify overfitting.

Possible Causes:

- **Overfitting:** The tree may memorize training examples instead of learning general patterns.
- **Deep tree structure:** Excessive depth creates highly specific decision rules.
- **Noisy data:** Incorrect or inconsistent observations can produce unnecessary branches.
- **Too many features:** Irrelevant attributes may result in poor generalization.
- **Insufficient training data:** A small dataset may not represent unseen cases effectively.

DATASET:

Sample	Age	Income	Student	Buys Computer
1	22	Low	Yes	Yes
2	25	Low	No	No
3	28	Medium	Yes	Yes
4	35	High	No	No
5	40	High	Yes	Yes
6	45	Medium	No	No
7	30	Medium	Yes	Yes
8	50	High	No	No

CODE:

```
from sklearn.tree import DecisionTreeClassifier
```

```
X = [  
    [22, 0, 1],  
    [25, 0, 0],  
    [28, 1, 1],  
    [35, 2, 0],  
    [40, 2, 1],  
    [45, 1, 0],  
    [30, 1, 1],  
    [50, 2, 0]
```

```
]
```

```
y = [  
    "Yes",
```

```
    "No",
```

```
    "No",
```

```

    "Yes",
    "No",
    "Yes",
    "No",
    "Yes",
    "No"
]

# Create Decision Tree

model = DecisionTreeClassifier(

    max_depth=None,

    random_state=42

)

# Train model

model.fit(X, y)

# Training prediction

train_prediction = model.predict(X)

print(
    "Training Predictions:"
)

print(train_prediction)

# Training accuracy

accuracy = model.score(X, y)

print(
    "\nTraining Accuracy:",
    accuracy
)

# Create Pruned Tree

pruned_model = DecisionTreeClassifier(

```

```
max_depth=3,
min_samples_split=2,
random_state=42
)
# Train pruned tree
pruned_model.fit(X, y)
print(
    "\nPruned Tree Accuracy:",
    pruned_model.score(X, y)
)
# Unseen test data
test_data = [
    [32, 1, 1],
    [48, 2, 0]
]
# Prediction
prediction = pruned_model.predict(
    test_data
)
print(
    "\nTesting Unseen Data:"
)
print(prediction)
```

OUTPUT:



```
Run co2-at3 x
/Users/moshika/PycharmProjects/PythonProject/.venv/bin/python /Users/moshika/PycharmProjects/PythonProject/co2-at3.py
Training Predictions:
['Yes' 'No' 'Yes' 'No' 'Yes' 'No' 'Yes' 'No']
Training Accuracy: 1.0
Pruned Tree Accuracy: 1.0
Testing Unseen Data:
['Yes' 'No']
Process finished with exit code 0
```

Debugging Strategies:

- Compare training accuracy with validation or testing accuracy.
- Check the depth and number of nodes in the tree.
- Identify branches created from very few samples.
- Test the model using unseen examples.
- Examine irrelevant or noisy features.

Optimization Techniques:

- Apply **pre-pruning** using maximum tree depth.
- Set a minimum number of samples required for splitting.
- Use **post-pruning** to remove unnecessary branches.
- Apply cross-validation to select suitable tree parameters.
- Remove irrelevant features using feature selection.

Conclusion:

The experiment shows that a decision tree can achieve very high training accuracy while performing poorly on unseen data when it becomes excessively complex. Pruning, cross-validation, suitable depth selection, and feature selection reduce overfitting and improve the model's ability to generalize.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	20	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	30	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	10	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand